

Software Engineering Fundamentals

Xây dựng Mental Model chuẩn xác

CLB TCC Học Thuật

3 buổi x 3 tiếng = 9 tiếng bút phá

Lộ trình 3 buổi

Chúng ta sẽ đi từ đầu đến cuối một request

Trình duyệt → Network → Server → Container → OS/Kernel → Trả lời

Buổi	Chủ đề	Thời lượng
1	Hệ thống & Mạng	3 tiếng
2	Hạ tầng & Container	3 tiếng
3	Kiến trúc & Bảo mật	3 tiếng

Nội dung Buổi 1

Phá vỡ "hộp đen" của máy tính

Phần 1 (45 phút): Operating System & Kernel

Phần 2 (60 phút): Đi sâu vào Linux & Terminal

Nghỉ giải lao (15 phút)

Phần 3 (75 phút): Networking Fundamentals

Mục tiêu Buổi 1

Sau buổi này, bạn sẽ hiểu:

- Software chạy TRÊN cái gì
- Kernel Space vs User Space khác nhau thế nào
- System Call là gì và hoạt động ra sao
- Linux terminal và các lệnh cơ bản
- Máy tính nói chuyện với nhau qua mạng như thế nào

Operating System là gì?

Tại sao phần mềm không thể nói chuyện trực tiếp với phần cứng?

Vấn đề: CPU có thể đọc/ghi bất kỳ vùng nhớ nào

Nếu app của bạn có bug → xóa toàn bộ ổ cứng? → Toàn bộ hệ thống sập

Giải pháp: OS đứng giữa, kiểm soát mọi thao tác với phần cứng

"OS là người gác cổng duy nhất được phép nói chuyện với phần cứng"

OS làm gì thực sự?

7 chức năng cốt lõi của Kernel

Chức năng	Chi tiết	Ví dụ
Process Management	Tạo, lên lịch, dừng process	Scheduler quyết định process nào chạy khi nào
Memory Management	Cấp phát RAM cho từng process	Virtual memory - mỗi app nghĩ nó có RAM riêng
File System	Tổ chức dữ liệu trên đĩa	Đọc/ghi file an toàn
Device Management	Giao tiếp với phần cứng	Drivers cho keyboard, GPU, network card
Security & Permissions	Ai được làm gì?	File permissions (rwx), user/group
Networking	TCP/IP stack	Gửi packet ra internet
IPC	Processes giao tiếp với nhau	Pipes, sockets, shared memory

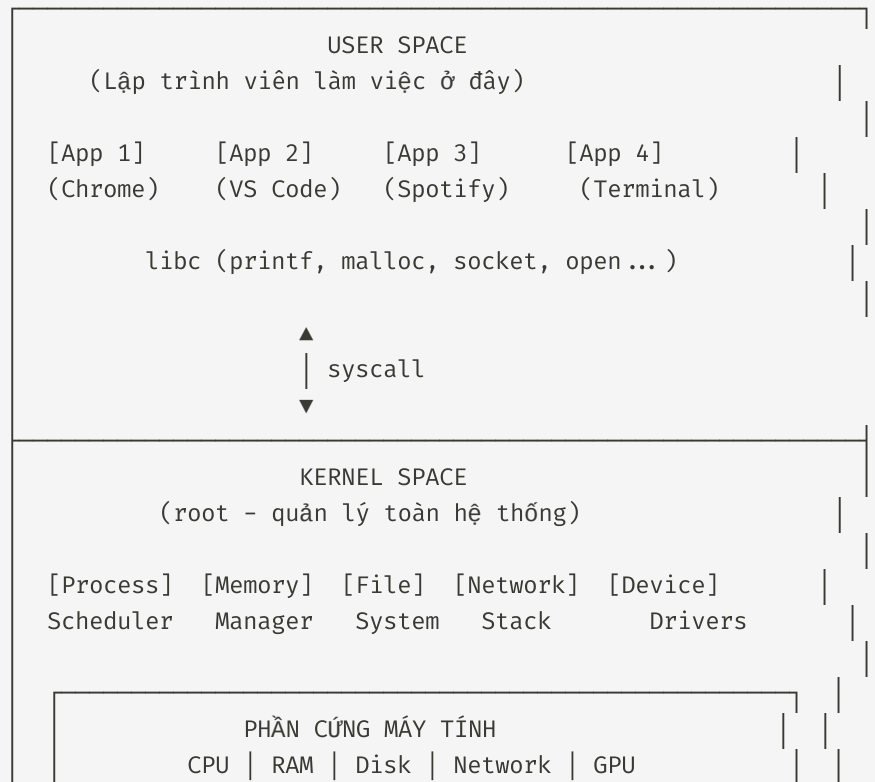
Khi bạn mở Chrome, điều gì xảy ra?

Từ click đến hiển thị - 8 bước chi tiết

1. Click icon Chrome
 - |
2. Desktop Environment (GNOME/KDE) nhận event
 - Gửi message đến process "Chrome"
 - |
3. Kernel kiểm tra: "Chrome được phép chạy không?"
 - User có quyền? Đủ RAM không? File executable?
 - |
4. Kernel tải Chrome binary từ disk vào RAM
 - Cấp phát virtual memory
 - Thiết lập namespaces (PID, network, mount...)
 - |
5. Chrome bắt đầu chạy trong User Space
 - Tạo renderer processes, GPU process
 - |
6. Chrome muốn vẽ cửa sổ → gọi syscall
 - syscall ioctl() → Kernel vẽ lên framebuffer
 - |
7. GPU nhận lệnh vẽ từ Kernel
 - |
8. Pixel trên màn hình!

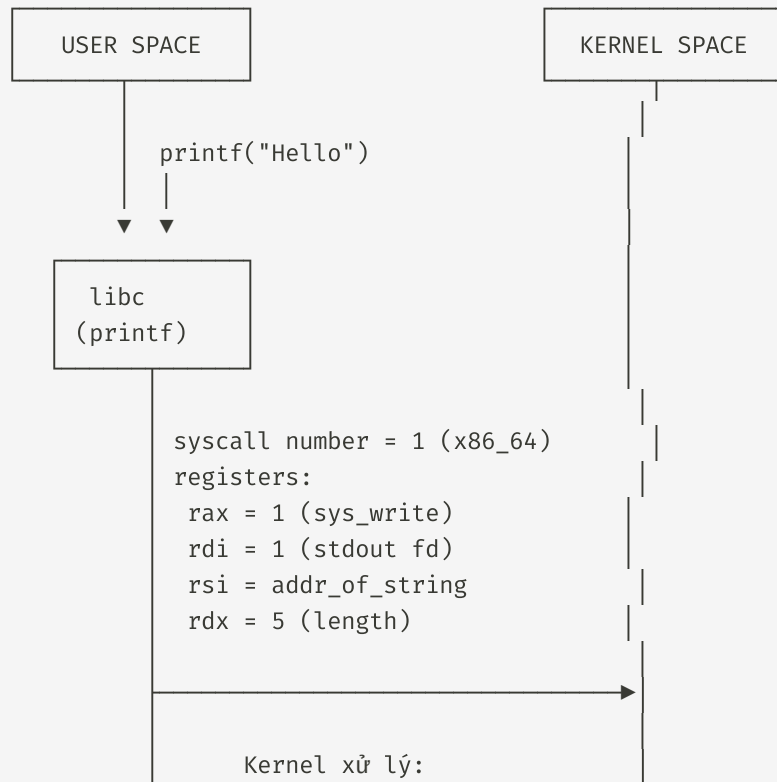
User Space vs Kernel Space

Hai vùng lãnh thổ của hệ thống



Khi gọi `printf("Hello")` - Chi tiết

Luồng syscall hoàn chỉnh



System Calls - Cầu nối giữa 2 thế giới

Những syscall quan trọng nhất cần nhớ

Syscall	Số (x86_64)	Chức năng	Giải thích
<code>read</code>	0	Đọc từ fd	fd có thể là file, stdin, socket
<code>write</code>	1	Ghi ra fd	Ghi ra terminal, file, network
<code>open</code>	2	Mở file	Trả về file descriptor
<code>close</code>	3	Đóng fd	Giải phóng file descriptor
<code>fork</code>	57	Tạo process mới	Copy process cha
<code>execve</code>	59	Chạy chương trình khác	Thay thế process image
<code>mmap</code>	9	Ánh xạ bộ nhớ	Cấp phát RAM hoặc map file

File Descriptor - Mỗi process có bảng riêng

fd = số nguyên đại diện cho "file" đang mở

Process "bash" - PID 1234

- fd 0: stdin → /dev/pts/0 (terminal input)
- fd 1: stdout → /dev/pts/0 (terminal output)
- fd 2: stderr → /dev/pts/0 (error output)
- fd 3: /etc/passwd → opened by login
- fd 4: ~/.bashrc → opened by bash
- fd 5: /tmp/socket → IPC socket
- fd 6: /var/log/syslog ► opened by logger

"Tất cả đều là file" - thiết bị, process, socket đều là fd!

Demo: System Calls với `strace`

Nhìn thấy "hơi thở" của chương trình

```
# Theo dõi tất cả syscalls
strace ls -la /tmp

# Output thực tế:
# execve("/usr/bin/ls", ["ls", "-la", "/tmp"], 0x7ffd... ) = 0
# brk(NULL)                                = 0x55a3b2c00000
# access("/etc/ld.so.preload", R_OK)       = -1 (ENOENT)
# openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY) = 3
# newfstatat(AT_FDCWD, "/tmp", { ... }, 0) = 0
# getdents64(3, [ ... ], 1024)            = 48
# write(1, "total 8\ndrwxrwxrwt  2 ...", 41) = 41
# +++ exited with 0 +++

# Count syscalls - thống kê
strace -c ls -la /tmp

# Filter chỉ syscalls cụ thể
strace -e trace=open,read,write cat /etc/hostname

# Trace child processes (fork + exec)
strace -f npm run dev
```

Demo: strace - Phân tích output

Mỗi dòng = một syscall được gọi

```
execve("/usr/bin/ls", ["ls", "-la", "/tmp"], 0x7ffd... ) = 0
```

Command + args Environment vars Return value = 0 (success)

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
```

Directory fd Path to file
(AT_FDCWD = current dir)

Return value = 3
(fd = 3)

```
write(1, "total 8\ndrwxr-xr-x 2 root root ... ", 48) = 48
```

File descriptor (1 = stdout)

Data written

Bytes written

Return: số bytes thực sự ghi được

-1 (ENOENT) = lỗi: file không tồn tại

-1 (EACCES) = lỗi: permission denied

Linux: Vì sao Software Engineer phải biết?

96.3% server trên thế giới chạy Linux

THỰC TẾ NGÀNH 2026

Server Market Share

- Linux: 96.3%
- Windows Server: 3.2%
- Others: 0.5%

Cloud Providers (tất cả đều Linux)

- AWS EC2: Linux AMIs chiếm ~80%
- GCP Compute Engine: 75%+ Linux
- Azure VMs: 60%+ Linux

Container Ecosystem

- Docker: Linux-based
- Kubernetes: Linux nodes
- Serverless (Lambda): Linux runtime

Job Market

- Backend jobs: Linux là yêu cầu gần như bắt buộc
- DevOps/SRE: Linux + bash scripting = cơ bản

File System Hierarchy

"Everything is a file" - Mọi thứ đều là file

```
/ (root - thư mục gốc của mọi thứ)
├── bin/      (essential binaries - ls, cp, mv, rm, cat)
├── sbin/     (system binaries - chỉ root chạy - fdisk, mkfs)
├── etc/      (configuration - passwd, shadow, nginx/, systemd/)
│   ├── passwd → danh sách users
│   ├── shadow → hashed passwords (root only!)
│   └── nginx/nginx.conf
├── home/     (user home directories)
│   └── khang/
│       ├── Documents/
│       └── Projects/
├── var/      (variable data - log, cache, web)
│   ├── log/
│   │   ├── syslog      (system log)
│   │   ├── auth.log    (login attempts)
│   │   └── nginx/
│   │       ├── access.log
│   │       └── error.log
│   └── www/html/       (web root)
```

Giải thích chi tiết từng thư mục

Tại sao /proc không tồn tại trên đĩa?

```
# Kiểm tra: proc mount point
df -h | grep proc
# proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)

# /proc là filesystem ẢO - không có trên disk!
# Kernel expose thông tin qua filesystem này

# Xem thông tin process
cat /proc/1/cmdline      # Command line đã chạy
cat /proc/1/environ      # Environment variables
ls -la /proc/1/fd/       # File descriptors đang mở

# CPU info
cat /proc/cpuinfo

# Memory info
cat /proc/meminfo

# Mỗi số trong /proc/ tương ứng với 1 process đang chạy!
```


Processes & Threads

Hai đơn vị thực thi trong hệ thống

Process "Chrome" (PID 1234)



Đặc điểm

Process

Thread

Process Lifecycle

Từ fork() đến exit()



Demo: Quản lý Processes

Các lệnh cần nhớ

```
# Xem processes đang chạy
ps aux | head -20      # BSD style
ps -ef                 # System V style

# Tìm process cụ thể
ps aux | grep nginx

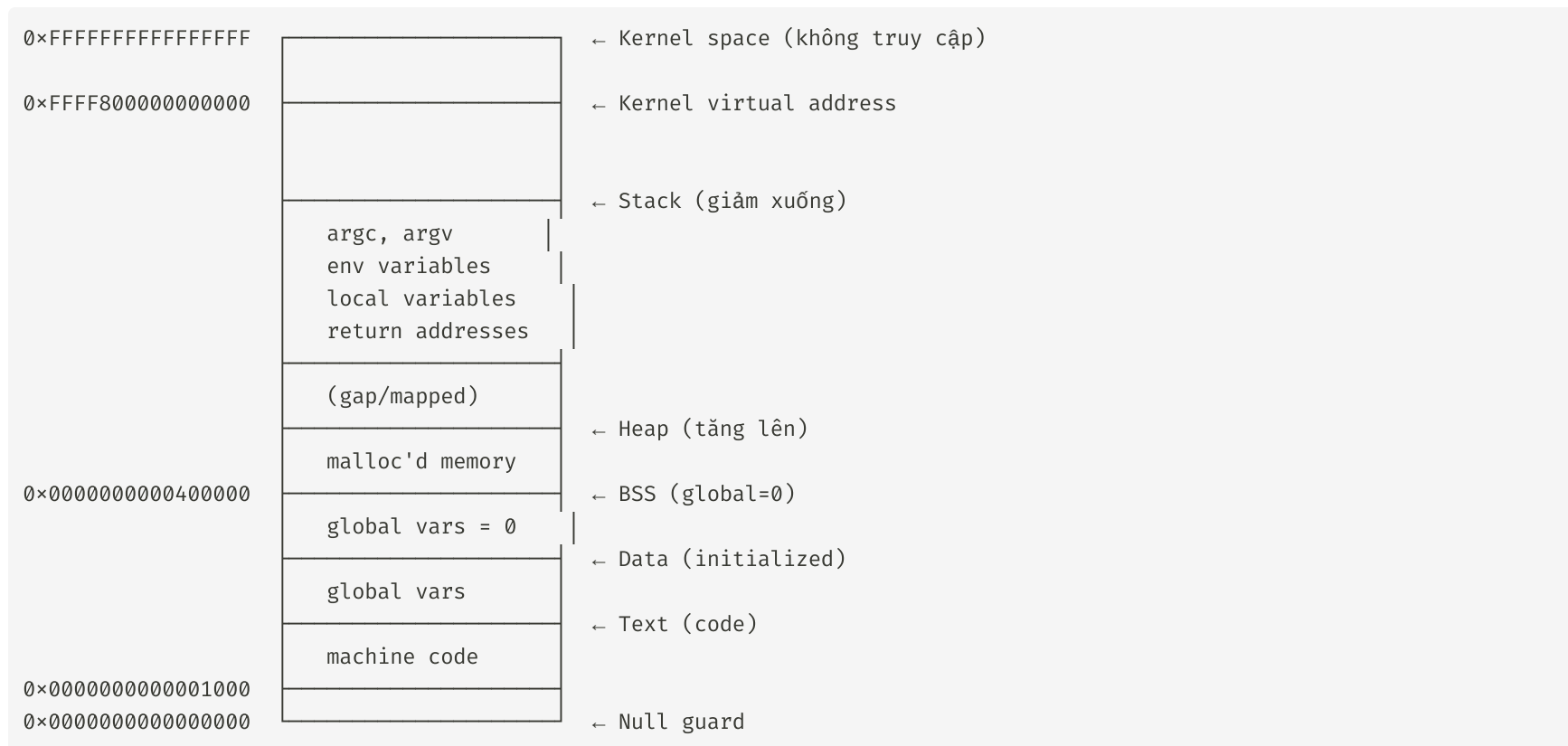
# Process tree
pstree                 # Tree view
pstree -p              # Với PIDs

# Realtime monitoring
top                    # Phím: M=Memory, P=CPU, k=kill, r=renice
htop                   # Giao diện đẹp hơn (cần cài)

# Kill process
kill PID               # SIGTERM (graceful shutdown)
kill -9 PID            # SIGKILL (buộc dừng ngay)
kill -15 PID           # SIGTERM (tương đương kill thường)
kill -STOP PID         # Pause process
kill -CONT PID         # Resume process
```

Process Memory Layout

Mỗi process có không gian địa chỉ ảo riêng



Ai được làm gì với file nào?

```
# Xem quyền: rwxr-xr--
# r = read (4), w = write (2), x = execute (1)
#      Owner      Group    Others

ls -la myfile.txt
# -rw-r--r--   1 khang  staff  4096   May  8 14:00  myfile.txt
# ┌──┬──┬──┐
# │   │   │   └─ Others (r-- = read only)
# │   └───┬───┘ Group (r-- = read only)
# └──────┴───┘ Owner (rw- = read + write)

# Chmod numeric - mỗi số = tổng r(4)+w(2)+x(1)
chmod 755 script.sh      # rwxr-xr-x (7=owner, 5=group, 5=others)
chmod 644 file.txt       # rw-r--r-- (6=owner, 4=group, 4=others)
chmod 600 secret.key     # rw----- (6=owner, 0=group, 0=others)
chmod 4711 binary        # rws---x--x (setuid bit!)

# Chmod symbolic
chmod u+x file.sh        # Thêm execute cho owner
chmod g-w file.txt       # Bỏ write cho group
chmod a+x script.sh      # Thêm execute cho all
chmod u=rw,go=r file     # Set owner=rw, others=readonly
```

ACL - Access Control Lists

Chi tiết hơn rwx - kiểm soát theo user cụ thể

```
# Xem ACL
getfacl file.txt

# Thêm user cụ thể
setfacl -m u:khang:rw file           # kang có read+write
setfacl -m u:bob:r file              # bob chỉ có read
setfacl -m g:staff:rx dir/           # group staff có read+execute

# Xóa ACL
setfacl -x u:khang file              # Xóa ACL entry của kang

# Set mask (giới hạn max permissions)
setfacl -m m::rx file                # Max permissions = r-x

# Default ACL (tự động áp dụng cho file mới trong directory)
setfacl -m d:u:visitor:r dir/        # File mới trong dir: visitor đọc được
```

Networking: OSI vs TCP/IP

Hai mô hình tham chiếu - Đừng học thuộc, hãy hiểu!

MÔ HÌNH OSI (7 TẦNG)		
Tầng 7	Application	HTTP, FTP, SMTP, DNS
Tầng 6	Presentation	SSL/TLS, JSON, ASCII
Tầng 5	Session	NetBIOS, RPC
Tầng 4	Transport	TCP (tin cậy), UDP (nhANH)
Tầng 3	Network	IP (định tuyến)
Tầng 2	Data Link	Ethernet, WiFi (MAC addresses)
Tầng 1	Physical	Cáp mạng, tín hiệu điện
TCP/IP (4 TẦNG)		
Tầng 4	Application	HTTP, DNS, SSH, FTP
Tầng 3	Transport	TCP, UDP
Tầng 2	Internet	IP, ICMP, ARP
Tầng 1	Link	Ethernet, WiFi

MẸO NHỚ: "A-P-S-N-T-N-L" → Application → Physical

Hoặc: "Please Do Not Throw Sausage Pizza Away"

(P)hysical-Data-Network-Transport-Session-Presentation-Application

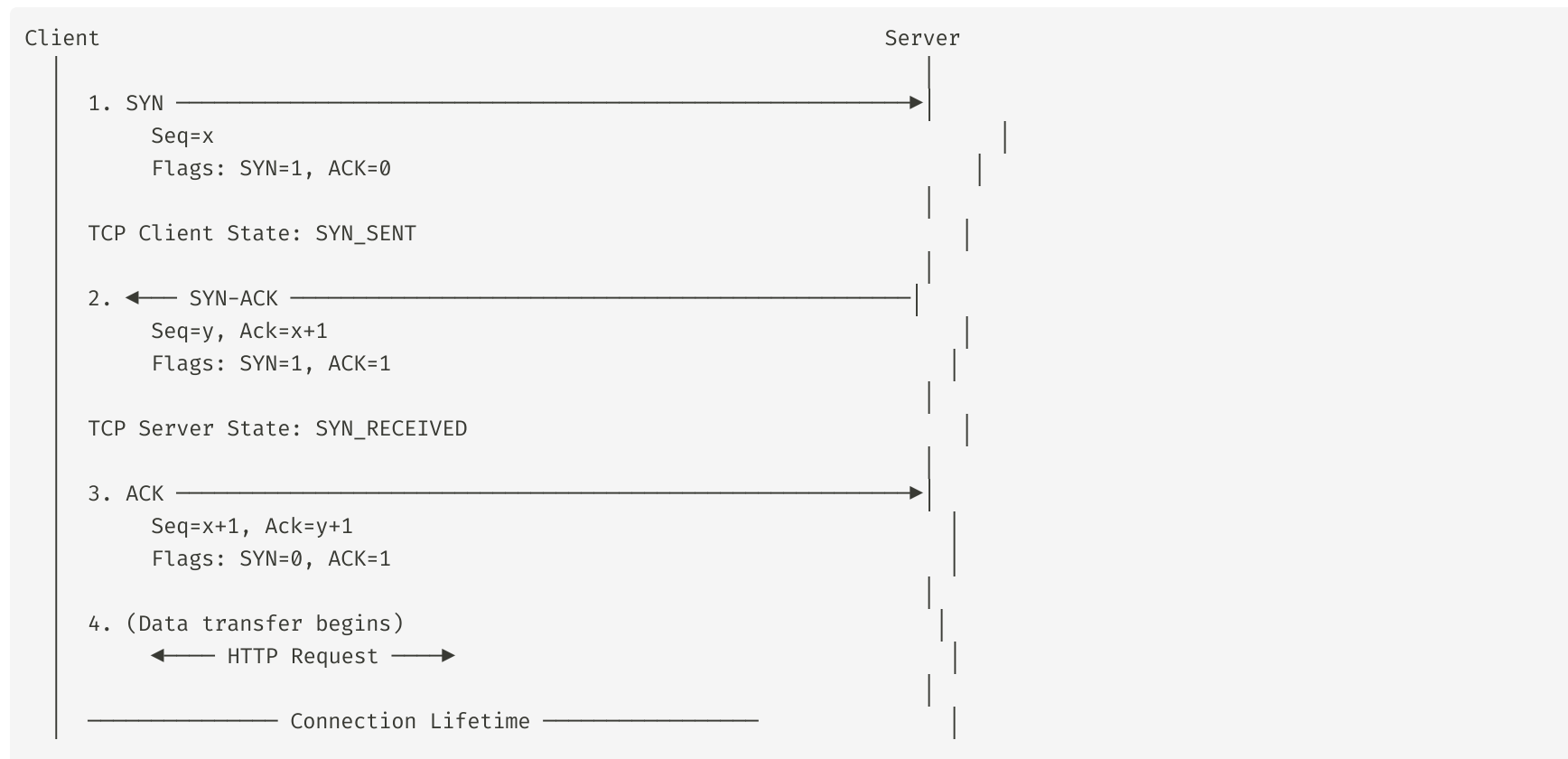
TCP vs UDP

Hai giao thức Transport - "Anh em sinh đôi khác tính"

Đặc điểm	TCP	UDP
Độ tin cậy	Đảm bảo đến đúng	Không đảm bảo
Thứ tự packet	Giữ đúng thứ tự	Không đảm bảo
Tốc độ	Chậm hơn (overhead)	Nhanh hơn
Connection	3-way handshake	Connectionless
Header	20-60 bytes	8 bytes (60% nhẹ hơn)
Flow control	Có (sliding window)	Không
Congestion control	Có	Không

TCP 3-Way Handshake

Chi tiết từng bước



UDP Header vs TCP Header

So sánh kích thước và cấu trúc

UDP Header (8 bytes - tối thiểu):

Source Port 16 bits	Dest Port 16 bits	Length 16 bits	Checksum 16 bits
------------------------	----------------------	-------------------	---------------------

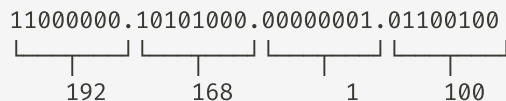
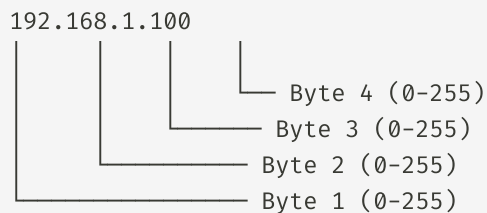
TCP Header (20-60 bytes):

Source Port 16 bits		Dest Port 16 bits													
Sequence Number 32 bits															
Acknowledgment Number 32 bits															
Offset 4bit	Resv 6bit	Flags 6bit	Window Size 16 bits			Checksum 16bits			Urgent 16bits						
		URG/ACK PSH/RST SYN/FIN													

IP Address

Địa chỉ nhà của máy tính trên mạng

IPv4: 4 bytes = 32 bits = 4.29 tỷ địa chỉ (ĐÃ HẾT!)



IPv6: 16 bytes = 128 bits = 340 undecillion địa chỉ

2001:0db8:85a3:0000:0000:8a2e:0370:7334
(viết gọn: 2001:db8:85a3::8a2e:370:7334)

Loại IPv4:

- Public: visible từ internet (1.1.1.1, 8.8.8.8)
- Private: chỉ trong mạng local
 - 10.0.0.0/8 (10.x.x.x)

Port - Số phòng trên máy tính

Xác định ứng dụng cụ thể

Port 16 bits (0-65535)

Well-known Ports (0-1023) → Chỉ root mới bind được

— 22	SSH	(Secure Shell)
— 25	SMTP	(Email sending)
— 53	DNS	(Domain Name System)
— 80	HTTP	(Web server)
— 443	HTTPS	(Secure Web)
— 3306	MySQL	
— 5432	PostgreSQL	

Registered Ports (1024-49151) → Dùng cho ứng dụng

— 3000	Node.js dev server
— 5173	Vite dev server
— 8000	Django dev server
— 8080	HTTP alternate / Tomcat

Dynamic/Private (49152-65535) → OS tự chọn khi connect()

Ví dụ: Server A có IP 203.0.113.10

DNS - Danh bạ của internet

Tên miền → IP Address

DNS Resolution Flow - 3 bước

1. Client → "example.com là gì?"
 - 2. Recursive Resolver (8.8.8.8 hoặc DNS của ISP)
 - Bước 1: Hỏi Root Nameserver
"ai quản lý .com?"
→ Root NS trả lời: "Hỏi .com TLD NS này"
 - Bước 2: Hỏi .com TLD Nameserver
"ai quản lý example.com?"
→ TLD NS trả lời: "Hỏi Authoritative NS này"
 - Bước 3: Hỏi Authoritative Nameserver
"IP của example.com là gì?"
→ Authoritative NS: "93.184.216.34"
3. Client nhận: example.com → 93.184.216.34
(Cache kết quả - TTL thường 300-86400 giây)

Các loại DNS Record

Bảng tra cứu nhanh

Record	Mô tả	Ví dụ
A	IPv4 address	<code>example.com → 93.184.216.34</code>
AAAA	IPv6 address	<code>example.com → 2606:2800:220:1::</code>
CNAME	Canonical name (alias)	<code>www.example.com → example.com</code>
MX	Mail exchange	<code>example.com → mail.example.com</code>
TXT	Text data	SPF, DKIM, domain verification
NS	Nameserver	<code>example.com → ns1.example.com</code>
PTR	Reverse DNS	<code>1.2.3.4 → hostname.in-addr.arpa</code>

Demo: Trace một HTTP Request

Từ trình duyệt đến server - 5 bước

```
# 1. DNS Resolution
nslookup example.com
# Server: 8.8.8.8
# Address: 93.184.216.34

# 2. Ping - kiểm tra kết nối
ping -c 4 example.com

# 3. Traceroute - xem đường đi packet
traceroute example.com
# 1. 192.168.1.1      1.2ms    (gateway)
# 2. 10.0.0.1        5.3ms    (ISP)
# 3. 72.14.215.85    10.2ms   (backbone)
# 4. 93.184.216.34   11.5ms   (destination)

# 4. HTTP Request chi tiết
curl -v https://example.com

# * Host example.com:443 was resolved.
# * Connected to example.com (93.184.216.34) port 443
# * TLS handshake
# > GET / HTTP/2
```

HTTP Request - Cấu trúc chi tiết

Mỗi request đều có format nhất định

```
GET /api/users?page=1&limit=10 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
Content-Type: application/json
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Cookie: session_id=abc123; theme=dark
Cache-Control: no-cache
```

```
{
  "name": "Khang",
  "email": "khang@example.com"
}
```

- Request Line: METHOD + PATH + VERSION
- Headers: Metadata về request
 - Host: Server target
 - Accept: Loại response mong muốn
 - Authorization: Credentials

HTTP Response - Cấu trúc chi tiết

Server trả về gì?

```
HTTP/1.1 200 OK
Date: Fri, 08 May 2026 10:30:00 GMT
Server: nginx/1.24.0
Content-Type: application/json; charset=utf-8
Content-Length: 512
Connection: keep-alive
X-Request-ID: abc-123-def
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 999
Cache-Control: max-age=300, public
ETag: "abc123"
Strict-Transport-Security: max-age=31536000
```

```
{
  "data": [
    {
      "id": 1,
      "name": "Khang",
      "email": "khang@example.com"
    }
  ],
  "meta": {
```

Tóm tắt Buổi 1

Những điểm cần nhớ (vĩnh viễn)

MENTAL MODEL - BUỔI 1

1. USER SPACE ↔ KERNEL SPACE
Syscall = cầu nối duy nhất
strace = công cụ nhìn thấy syscall
2. LINUX = CHUẨN NGÀNH
96% server, 100% container, Cloud = Linux
/bin, /etc, /var/log, /home, /proc
3. PROCESS = instance chương trình
Thread = đơn vị nhỏ hơn, chia sẻ memory
PID = định danh duy nhất
4. TCP = TIN CẬY, UDP = NHANH
TCP: 3-way handshake, ack, retransmit, ordered
UDP: gửi rồi quên, không guarantee
5. IP + PORT = TÌM ĐÚNG MÁY + ĐÚNG ỨNG DỤNG
DNS: tên miền → IP

Câu hỏi ôn tập

Kiểm tra hiểu bài

1. Khi gọi `printf("hello")` trong C, có những layer nào tham gia?
2. Sự khác nhau giữa `fork()` và `execve()` ? Tại sao shell dùng cả hai?
3. Tại sao server web cần bind vào port 80/443, không phải port khác?
4. Làm thế nào để debug xem một chương trình gọi những syscall nào?
5. TCP handshake gồm những bước nào? Tại sao cần 3 bước?
6. Trình bày flow đầy đủ khi browser truy cập "https://google.com"

Buổi 2: Hạ tầng & Triển khai

Đưa ứng dụng lên production

Phần 1 (45 phút): Servers, Virtualization & Cloud

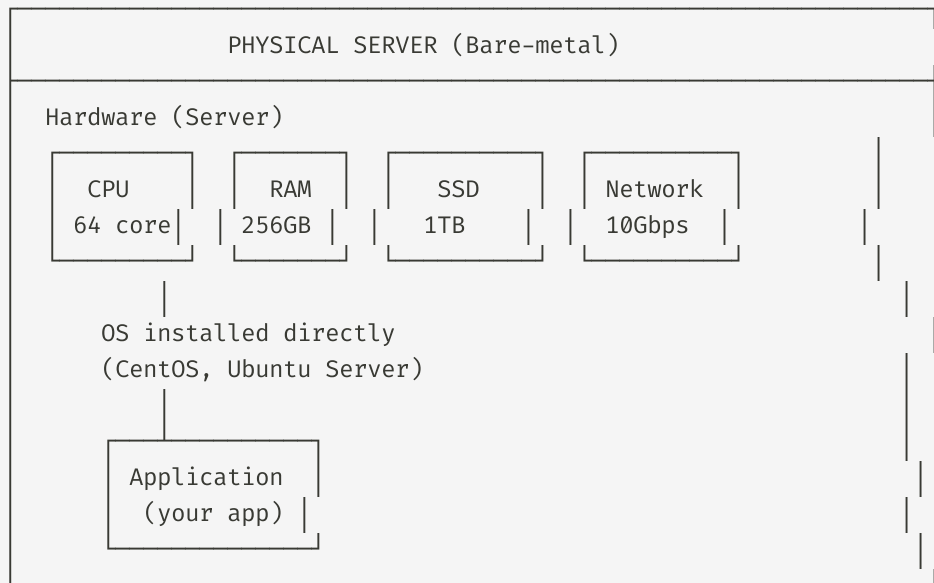
Phần 2 (45 phút): Containerization & Docker

Nghỉ giải lao (15 phút)

Phần 3 (75 phút): Web Servers & Demo tổng hợp

Physical Server vs Virtual Machine

Từ phần cứng thuần đến ảo hóa



Hypervisor

Phần mềm tạo máy ảo

Type 1: Bare-metal (chạy trực tiếp trên hardware)

Hardware → Hypervisor (VMware ESXi / KVM / Xen) → VMs
→ Hiệu suất cao, dùng trong datacenter
→ Ví dụ: VMware vSphere, Microsoft Hyper-V, KVM

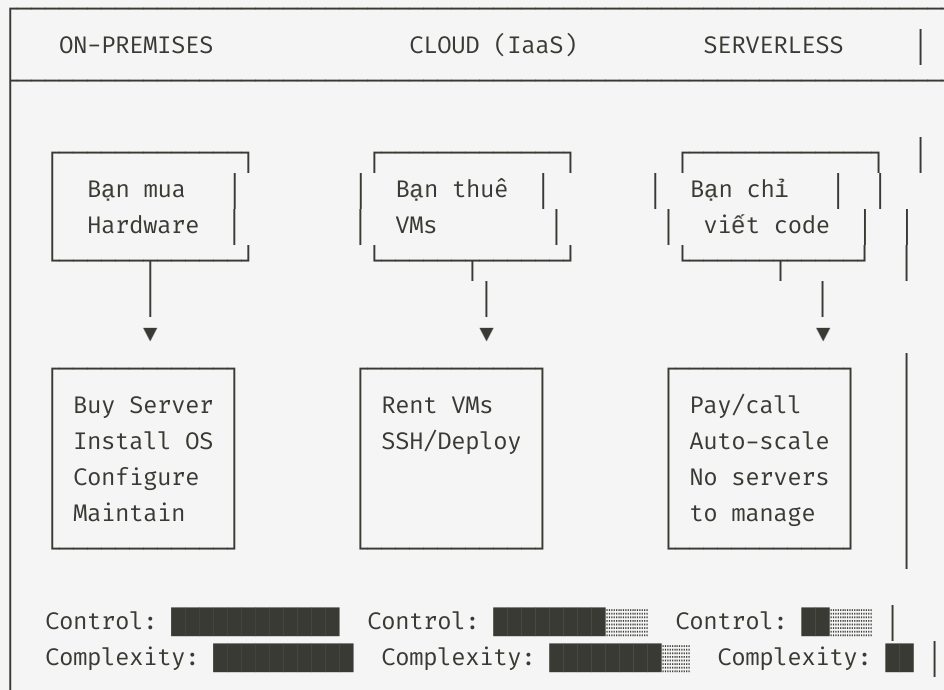
Type 2: Hosted (chạy trên OS thông thường)

OS (macOS/Windows) → Hypervisor (VirtualBox/VMware) → VMs
→ Dùng cho development/testing
→ Ví dụ: Oracle VirtualBox, VMware Workstation

Khía cạnh	Type 1	Type 2
Performance	Cao	Trung bình
Use case	Datacenter/Production	Development

Cloud Computing

Thuê tài nguyên thay vì mua



Vấn đề: "It works on my machine"

Nỗi đau của mọi developer

Năm 2010-2013: TRƯỚC DOCKER

Developer (macOS, Node 14)

```
$ node --version
v14.2.0
$ npm --version
6.14.4
$ python --version
3.8.2
```

Bug: "Database connection timeout"
Reason: mysql client version không tương

Production (Ubuntu 18.04, Node 18)

```
$ node --version
v18.16.0
$ npm --version
9.5.1
$ python --version
2.7.17
```

← Đêm khuya debug
production!

Năm 2013: DOCKER = GIẢI PHÁP HOÀN HẢO

Developer

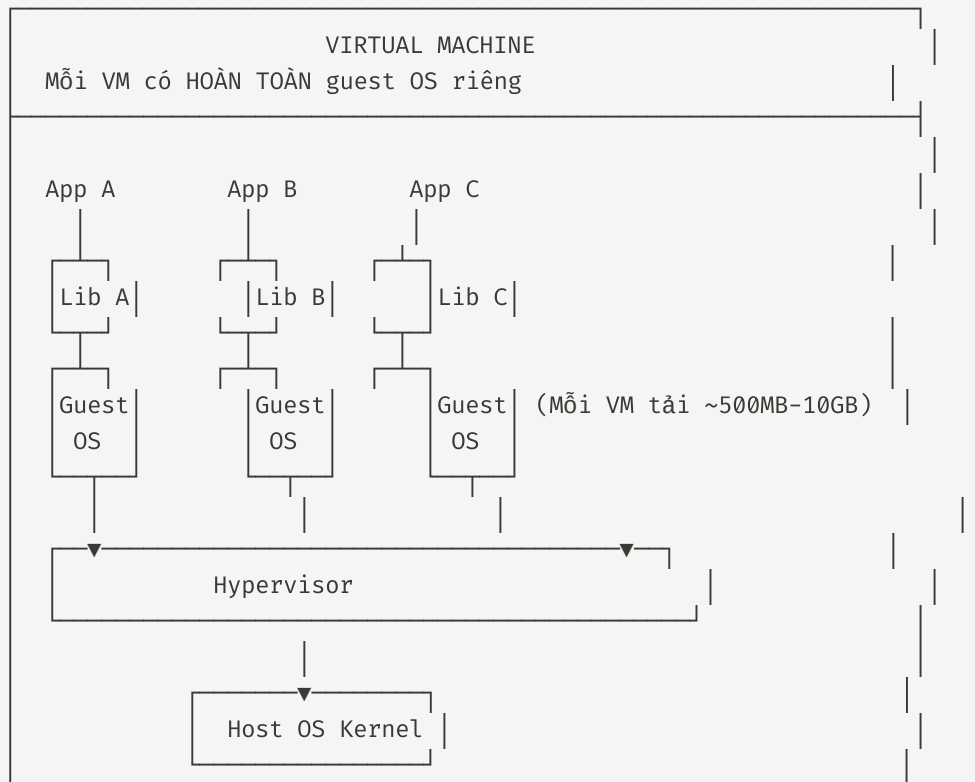
```
$ docker build -t myapp .
$ docker run myapp
```

Production

```
$ docker pull myapp
$ docker run myapp
```


Container vs Virtual Machine

Container không phải là VM!



Linux Namespaces

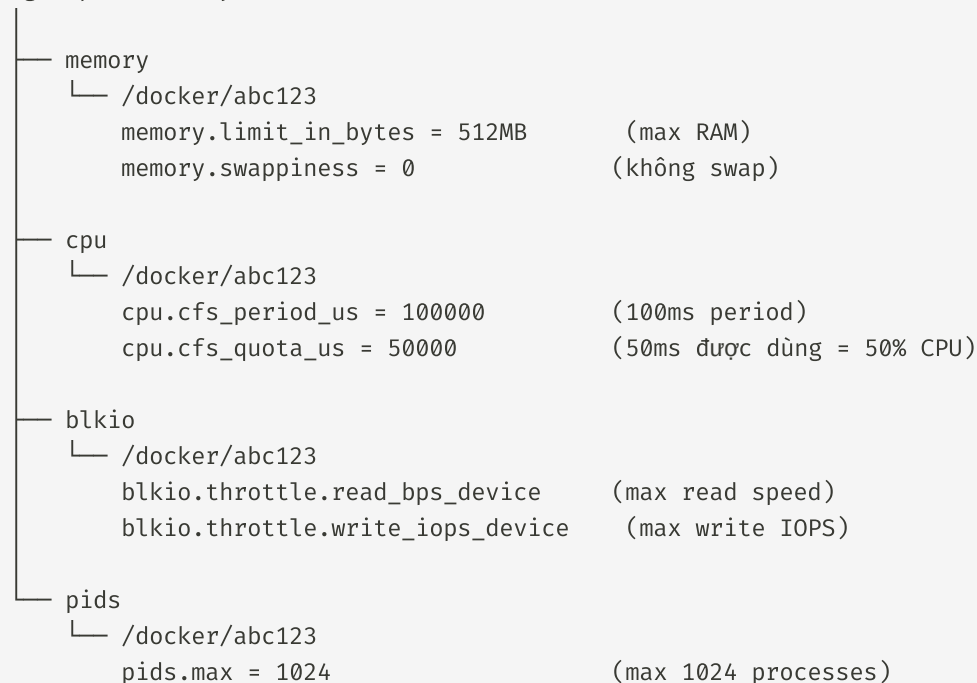
Công nghệ đằng sau Container

Namespace	Viết tắt	Cô lập gì	Ví dụ
PID	PID	Process IDs	Container thấy PID 1 là process của nó
Network	NET	Interfaces, ports	Container có IP riêng
Mount	MNT	Filesystem mount	Container có root filesystem riêng
User	USER	User/Group IDs	UID 0 (root) trong container $\neq$ host root
UTS	UTS	Hostname	Container có hostname riêng
IPC	IPC	Shared memory	Container không thấy SHM của container khác
Cgroup	CGROUP	Resource limits	Giới hạn CPU, RAM, I/O

Cgroups - Giới hạn tài nguyên

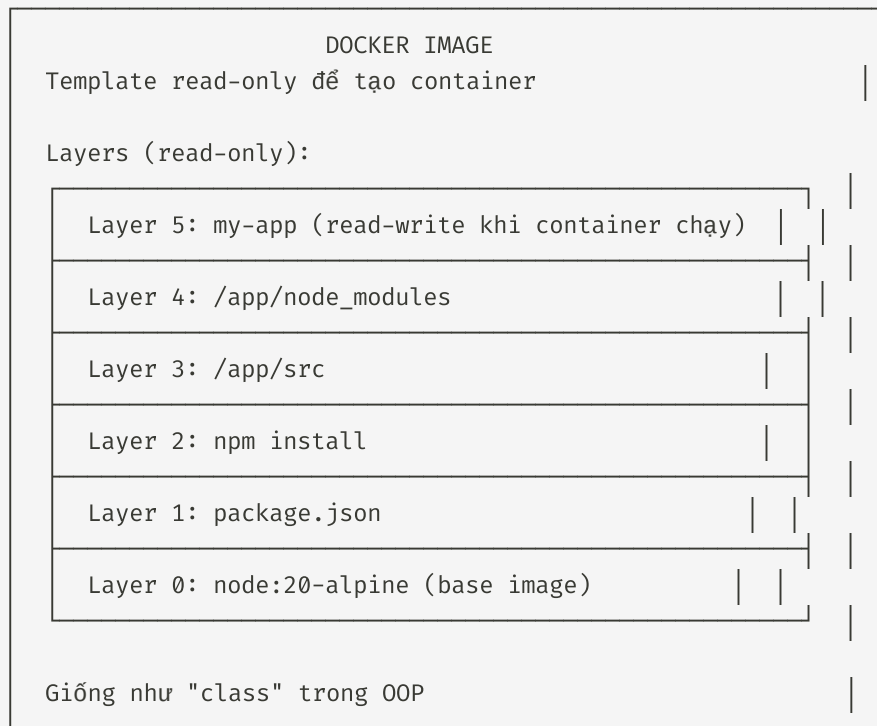
Container không thể dùng hết server resources

Cgroup hierarchy cho container



Docker: Image vs Container

Hai khái niệm cốt lõi



| docker run

Dockerfile

Build image từ Dockerfile - Multi-stage build

```
# =====  
# Stage 1: Builder (build production artifacts)  
# =====  
FROM node:20-alpine AS builder  
  
WORKDIR /app  
COPY package*.json ./  
RUN npm ci  
  
COPY . .  
RUN npm run build  
  
# =====  
# Stage 2: Production (runtime nhẹ)  
# =====  
FROM node:20-alpine AS production  
  
# Tạo non-root user cho bảo mật  
RUN addgroup -g 1001 -S nodejs && \  
    adduser -S nestjs -u 1001  
  
WORKDIR /app
```

Dockerfile Instructions

Từng lệnh và công dụng

Instruction	Mô tả	Tips
<code>FROM</code>	Base image	Luôn dùng version cụ thể: <code>node:20-alpine</code>
<code>COPY</code>	Copy file	<code>COPY package*.json ./</code> → cache layer
<code>RUN</code>	Chạy command	Gộp commands: <code>apt-get update && apt-get install</code>
<code>WORKDIR</code>	Set working directory	Tự tạo dir nếu chưa có
<code>ENV</code>	Environment variable	<code>ENV NODE_ENV=production</code>
<code>EXPOSE</code>	Khai báo port	Chỉ documentation

Docker Commands - Lifecycle

Build → Run → Manage

```
# Build và chạy
docker build -t myapp:1.0 .                # Build image
docker build -t myapp:1.0 -f Dockerfile.prod # Dockerfile khác
docker build --no-cache -t myapp .         # Không dùng cache

docker run -d -p 3000:3000 --name my-app myapp:1.0 # Chạy container
docker run -d -e NODE_ENV=production myapp        # Với env vars
docker run -d -v /host/path:/container/path myapp  # Với volume

# Xem layers của image
docker history myapp:1.0

# Lifecycle
docker ps                # Containers đang chạy
docker ps -a            # Tất cả containers
docker start/stop my-container # Start/stop
docker restart my-container  # Restart

# Truy cập container
docker exec -it my-container bash # Bash shell
docker exec my-container ls /app  # Chạy command
```

Docker Network

Container nói chuyện với nhau như thế nào?

```
bridge (default)
docker0: 172.17.0.1

Container A — eth0 (172.17.0.2)
Container B — eth0 (172.17.0.3)
Container C — eth0 (172.17.0.4) — docker0 bridge
                                   |
                                   veth — host eth0
```

Containers thấy nhau qua tên (Docker DNS)

```
host
Container dùng trực tiếp host network
Port 3000 trên container = port 3000 trên host
→ Phù hợp cho performance-critical apps
→ Mất isolation (không chạy 2 containers cùng port)
```

```
overlay (Docker Swarm)
```


Docker Compose

Quản lý multi-container app

```
version: "3.8"

services:
  api:
    build: ./backend
    ports:
      - "3000:3000"
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    networks:
      - app-net
    healthcheck:
      test: ["CMD", "wget", "--no-verbose", "--spider", "http://localhost:3000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  frontend:
    build: ./frontend
```

Forward Proxy vs Reverse Proxy

Hai loại proxy, hai mục đích

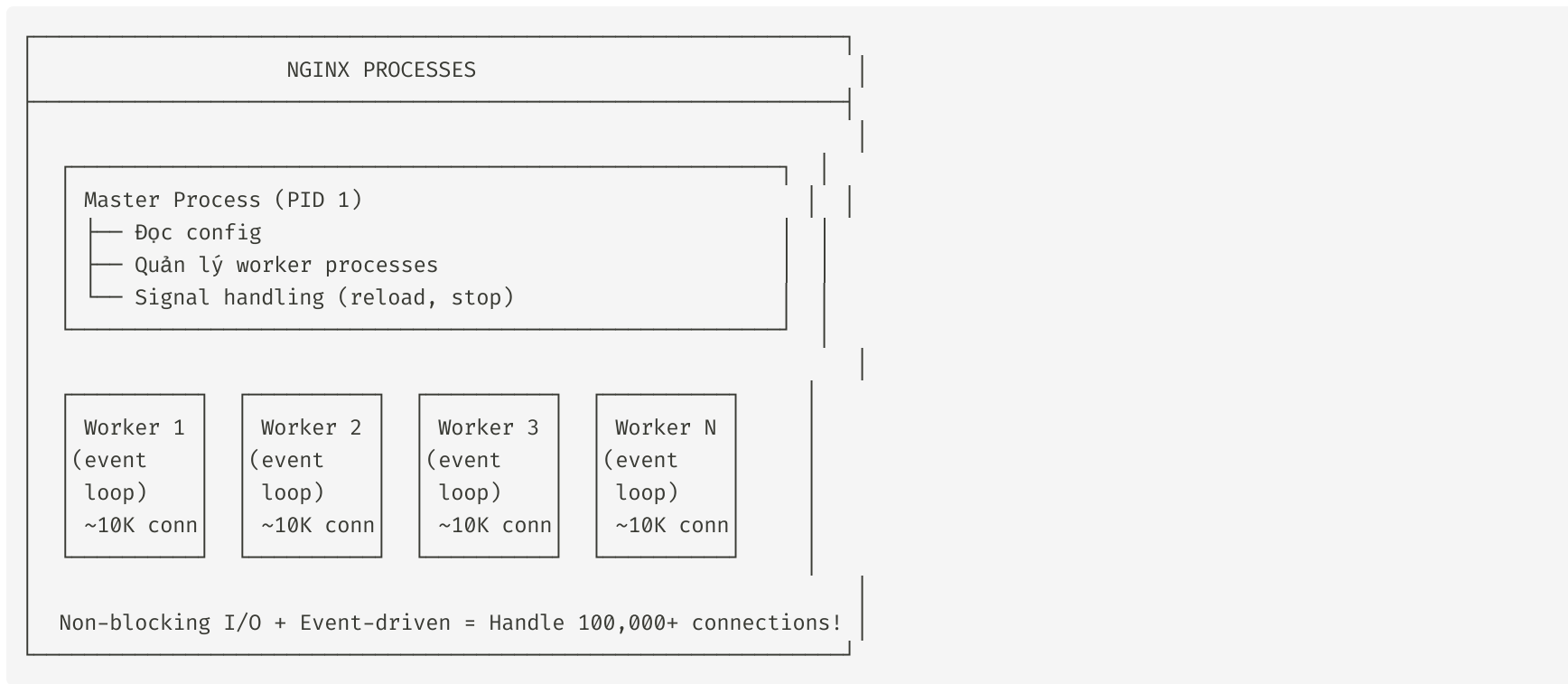
Forward Proxy (Proxy thuận):



Reverse Proxy (Proxy ngược):

Nginx: Reverse Proxy phổ biến nhất

Đứng trước ứng dụng, hứng traffic



Tại sao cần Nginx?

Nginx Configuration

Chi tiết upstream, location, và proxy settings

```
# upstream{} - Backend servers
upstream api_backend {
    least_conn;                # ít connections nhất

    server 172.17.0.2:3000;      # Container 1
    server 172.17.0.3:3000;      # Container 2
    server 172.17.0.4:3000;      # Container 3 down → loại bỏ tạm

    keepalive 32;               # Keep 32 idle connections
}

# rate limiting
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

server {
    listen 80;
    server_name example.com;

    # Redirect HTTP → HTTPS
    return 301 https://$server_name$request_uri;
}
```

Load Balancing Strategies

4 thuật toán phổ biến

```
# 1. Round Robin (mặc định)
upstream round_robin {
    server 172.17.0.2:3000;
    server 172.17.0.3:3000;
    server 172.17.0.4:3000;
    # Request 1 → server 1, Request 2 → server 2 ...
}

# 2. Least Connections
upstream least_conn {
    least_conn;
    server 172.17.0.2:3000 weight=3; # weight = capacity
    server 172.17.0.3:3000 weight=2;
}

# 3. IP Hash (session affinity)
upstream ip_hash {
    ip_hash;
    server 172.17.0.2:3000;
    server 172.17.0.3:3000;
    # user1 (IP A) → luôn server 1
    # user2 (IP B) → luôn server 2
```

Demo: Docker Compose + Nginx

Deploy NestJS + React + Nginx + PostgreSQL

docker-compose.yml:

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      api:
        condition: service_healthy
      frontend:
        condition: service_started

  api:
    build: ./backend
    environment:
      - DATABASE_HOST=postgres
      - DATABASE_PORT=5432
```

Tóm tắt Buổi 2

Những điểm cần nhớ

MENTAL MODEL - BUỔI 2

1. PHYSICAL → VM → CONTAINER

VM: Chia sẻ hardware qua Hypervisor

Container: Chia sẻ Kernel qua Namespaces + Cgroups

2. DOCKER = IMAGE + CONTAINER

Image = Template (class) | Container = Instance (object)

Dockerfile = Công thức nấu ăn | Layer = Mỗi instruction

3. DOCKER COMPOSE = Multi-container orchestration

Quản lý: build, network, volume, depends_on, healthcheck

4. NGINX = Reverse Proxy + Load Balancer

Event-driven architecture → handle 100K+ connections

upstream{} → load balancing | location{} → routing

5. NAMESPACES + CGROUPS = Container Isolation

Namespaces: PID, NET, MNT, USER, UTS, IPC

Cgroups: CPU, Memory, I/O limits

Buổi 3: Kiến trúc Ứng dụng & Bảo mật

Xây dựng phần mềm tương tác an toàn

Phần 1 (60 phút): Web Architecture & APIs

Phần 2 (30 phút): Database Fundamentals

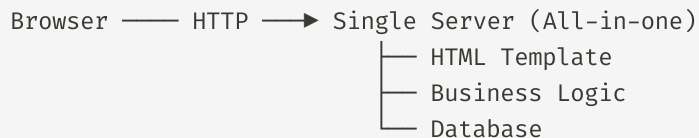
Nghỉ giải lao (15 phút)

Phần 3 (75 phút): Web Security

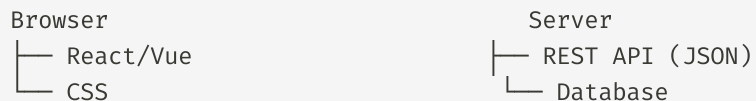
Evolution của Kiến trúc Web

Từ Monolith đến Microservices

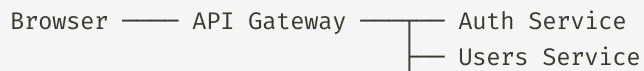
KỶ NGUYÊN 1: Monolith (2000-2010)



KỶ NGUYÊN 2: Client-Server (2010-2020)

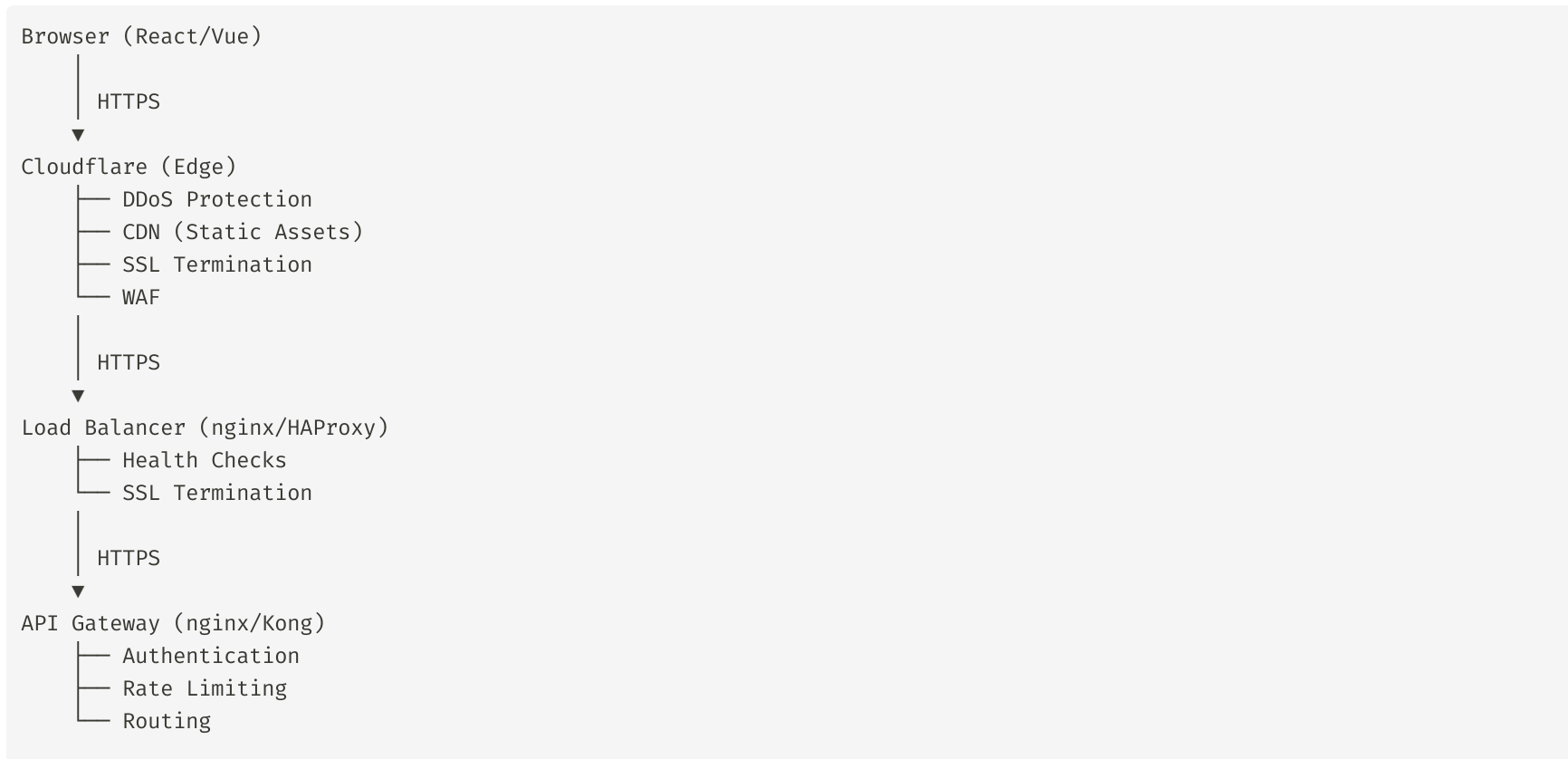


KỶ NGUYÊN 3: Microservices (2020+)



Modern Web Architecture

Chi tiết từ Browser đến Database



HTTP Methods

CRUD Operations trên resources

Method	CRUD	Mô tả	Idempotent	Safe
GET	Read	Lấy resource	Có	Có
POST	Create	Tạo resource mới	Không	Không
PUT	Replace	Thay thế toàn bộ	Có	Không
PATCH	Update	Cập nhật một phần	Không	Không
DELETE	Delete	Xóa resource	Có	Không
HEAD	-	Headers của resource	Có	Có
OPTIONS	-	Các methods hỗ trợ	Có	Có

HTTP Status Codes

Phản hồi từ server nói gì?

1xx: INFORMATIONAL

100 Continue — Client gửi tiếp body (Upload large file)

101 Switching — Upgrade protocol (HTTP → WebSocket)

2xx: SUCCESS

200 OK — Thành công thông thường

201 Created — Resource mới được tạo

204 No Content — Thành công, không có body

3xx: REDIRECTION

301 Moved — Chuyển vĩnh viễn (cache permanent)

302 Found — Chuyển tạm thời

304 Not Modified — Dừng cache (ETag/Last-Modified)

4xx: CLIENT ERROR (request sai - lỗi từ client)

400 Bad Request — Request sai cú pháp

401 Unauthorized — Chưa authenticate (chưa login)

403 Forbidden — Đã authenticate nhưng không có quyền

404 Not Found — Resource không tồn tại

409 Conflict — Conflict với state hiện tại

422 Unprocessable — Request đúng syntax nhưng sai logic

RESTful API Design

4 nguyên tắc cốt lõi

1. Uniform Interface:

TỐT:	XẤU:
GET /users	GET /getUsers
GET /users/123	GET /getUserById?id=123
POST /users	POST /createNewUser
DELETE /users/123	GET /deleteUser?id=123

2. Stateless:

MỖI REQUEST phải chứa TẤT CẢ thông tin cần thiết:

- Authentication token
- User context
- Pagination params

XẤU: Server lưu session → scaling khó

TỐT: Token self-contained → stateless

3. Cacheable:

REST vs GraphQL vs gRPC

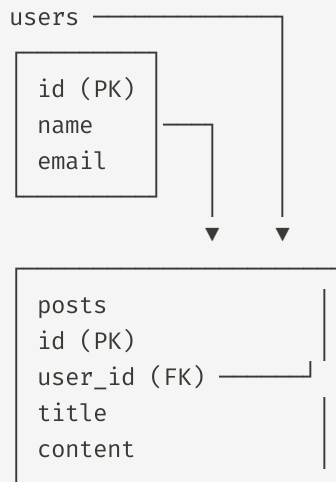
Khi nào dùng cái nào?

Khía cạnh	REST	GraphQL	gRPC
Dữ liệu	JSON/HTTP	JSON/HTTP	Protobuf
Real-time	Không	Không	Có (streaming)
Query flexibility	Cố định endpoint	Linh hoạt	Cố định
Performance	Trung bình	Chậm hơn	Rất nhanh (10-100x)
Browser native	Có	Có	Không
Caching	Dễ (URL)	Khó	Khó
Setup	Dễ	Trung bình	Phức tạp

SQL vs NoSQL

Chọn đúng loại database cho bài toán

SQL (PostgreSQL, MySQL):



- ✓ Schema cố định (ràng buộc chặt chẽ)
- ✓ ACID transactions (đảm bảo tính nhất quán)
- ✓ Complex queries với JOINS
- ✓ Horizontal scaling KHÓ (sharding phức tạp)

ACID Properties

Đảm bảo tính nhất quán của database

A - ATOMICITY (Tính nguyên tử)

Transaction = TẤT CẢ hoặc KHÔNG GÌ

Chuyển 500k từ tài khoản A → tài khoản B

```
BEGIN TRANSACTION
  UPDATE accounts SET balance = balance - 500
  UPDATE accounts SET balance = balance + 500
  IF error THEN ROLLBACK
  ELSE COMMIT
END TRANSACTION;
```

C - CONSISTENCY (Tính nhất quán)

Database luôn ở trạng thái VALID sau transaction
PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK constraints

I - ISOLATION (Tính cô lập)

ORM vs Raw SQL

Hai cách tương tác với database

ORM (Prisma - Type-safe):

```
// Query đơn giản
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: { posts: true }
})

// Query phức tạp
const users = await prisma.user.findMany({
  where: {
    AND: [
      { email: { endsWith: '@company.com' } },
      { posts: { some: { published: true } } }
    ]
  },
  orderBy: { name: 'asc' },
  take: 10,
  include: { _count: { select: { posts: true } } }
})
```

Authentication vs Authorization

Hai khái niệm dễ nhầm lẫn

AUTHENTICATION (Xác thực) - "BẠN LÀ AI?"

Xác minh identity của user

Methods:

- Password + Username
- OAuth 2.0 (Google, GitHub login)
- SSO (Single Sign-On)
- Biometric (fingerprint, face)
- Multi-factor (2FA: SMS, TOTP, Hardware key)

Result: Đăng nhập thành công → nhận token/session

AUTHORIZATION (Phân quyền) - "BẠN ĐƯỢC LÀM GÌ?"

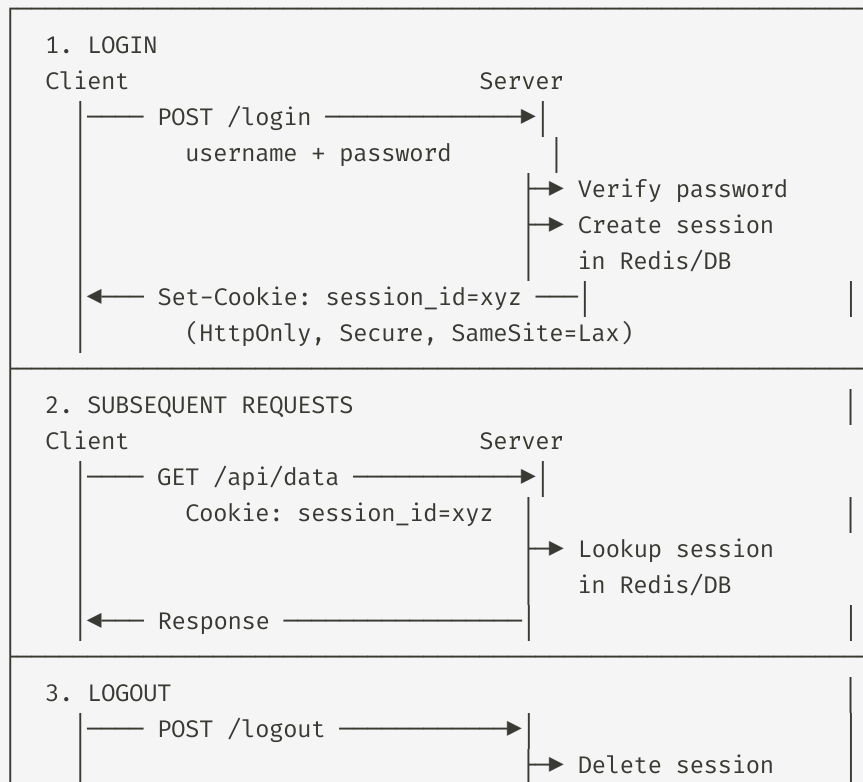
Kiểm tra quyền hạn của user đã authenticate

Models:

- RBAC (Role-Based)
 - admin: full access
 - editor: read + write

Session-based Authentication

Server lưu trạng thái user



JWT (JSON Web Token)

Token tự chứa thông tin

JWT STRUCTURE:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

Base64Url(Header)

{"alg": "HS256", "typ": "JWT"}

.eyJzdWIiOiIxMjM0NTY3ODkwIiwiaXhwIjozNzQ2 ...

Base64Url(Payload)

{"sub": "user_123", "role": "admin", "exp": ... }

.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

HMAC-SHA256(Header.Payload, Secret)

PAYLOAD (Claims):

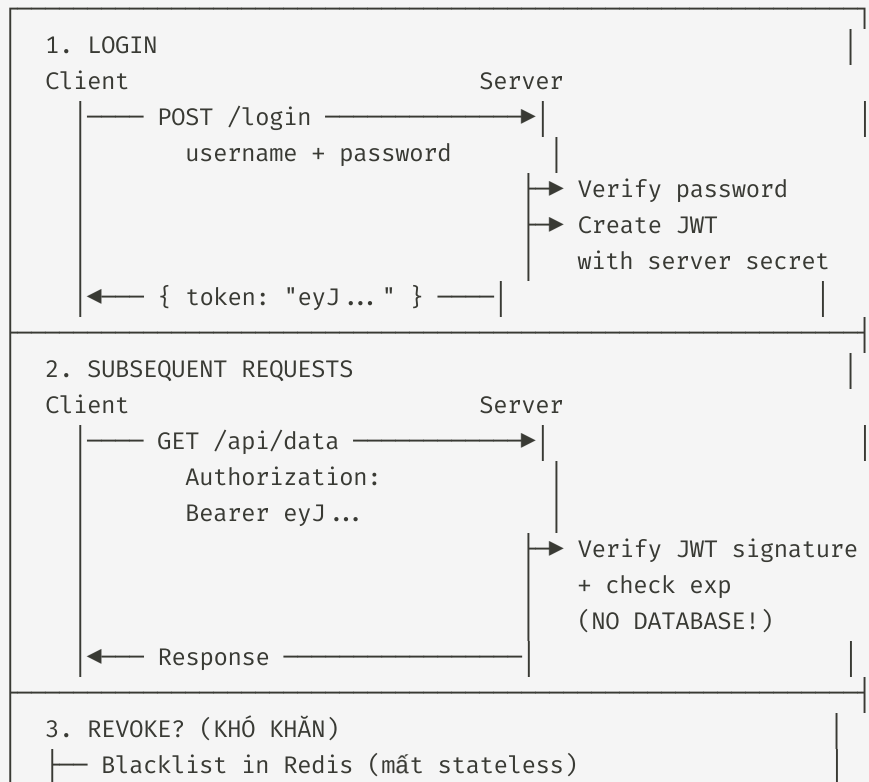
{

"sub": "user_123", // Subject (user ID)

"name": "Khang", // Tên user

JWT Flow

Từ login đến authenticated request



JWT Tamper Demo

Tại sao signature quan trọng?

```
// 1. Decode payload (không cần key)
const payload = atob('eyJzdWIiOiIxMjM0NTY3ODkwIiwiaWwiYWRTaW4iOmZhbHNlfQ==')
// {"sub":"1234567890","admin":false}

// 2. Thay đổi admin: false → admin: true
const fakePayload = btoa(JSON.stringify({
  sub: "1234567890",
  admin: true // ← Thay đổi ở đây!
}))

// 3. Tạo token mới: header.payload.signature
// Signature vẫn dùng secret cũ → KHÔNG HỢP LỆ!

// 4. Server verify signature → REJECTED!
// "JsonWebTokenError: invalid signature"
```

JWT Tamper = Payload thay đổi được, nhưng KHÔNG tạo được signature hợp lệ

CORS: Cross-Origin Resource Sharing

Tại sao trình duyệt chặn request?

SAME-ORIGIN POLICY

Origin = Protocol + Domain + Port

http://example.com:3000

↓ ↓ ↓
protocol domain port

Same Origin: ✓ http://example.com → http://example.com

Different Origin:

× http://localhost:3000 → http://localhost:5173

× http://example.com → http://api.example.com

× http://example.com → https://example.com

CORS WORKFLOW

Browser

Server

| — GET /api —> | (Simple request)

SQL Injection

Khi input của user trở thành câu lệnh SQL

VÍ DỤ THƯỜNG GẶP

BAD (concatenation):

```
const query = `SELECT * FROM users  
WHERE username = '${username}'  
AND password = '${password}'`;
```

ATTACK: username = "admin' --"

→ Query trở thành:

```
SELECT * FROM users WHERE username = 'admin' --'  
AND password = ''
```

→ '--' comment out → bypass login!

ATTACK: username = "'; DROP TABLE users; --"

→ Table users bị XÓA!

PHÒNG CHỐNG

GOOD (parameterized query):

```
pool.query(  
  'SELECT * FROM users WHERE username = $1 AND password = $2',
```


XSS: Cross-Site Scripting

Inject script độc hại vào trang web

3 LOẠI XSS

1. STORED XSS (Nguy hiểm nhất)

Malicious script được LƯU vào database

1. Attacker post: "Bài viết hay!" + `<script>stealCookies()`
2. Comment được lưu vào DB
3. User đọc comment → Browser execute script → cookie bị đánh cắp

2. REFLECTED XSS

Script nằm trong URL, được "phản chiếu" trong response

URL: `https://search.com?q=<script>alert(1)</script>`
→ Response: "Kết quả cho: `<script>alert(1)</script>`"

3. DOM-BASED XSS

Script được execute bởi JavaScript phía client

CSRF: Cross-Site Request Forgery

Lừa user gửi request không biết

ATTACK SCENARIO

1. User đã login ngân hàng.com (session valid)
2. User mở email từ attacker:
``
3. Browser tự động gửi:
 - Request đến nganhang.com
 - Kèm cookie session của user
 - Server không phân biệt được request hợp lệ
4. 10 triệu đi đâu?

PHÒNG CHỐNG:

- CSRF Token (mỗi form cần token unique)
`<input type="hidden" name="csrf_token" value="abc123">`
- SameSite Cookie

Security Headers

Defense in Depth

```
// Security Headers - Middleware cho Express
import helmet from 'helmet'

app.use(helmet())

// Chi tiết:

// 1. Content-Security-Policy (CSP)
// Ngăn chặn XSS
res.setHeader('Content-Security-Policy',
  "default-src 'self'; " +
  "script-src 'self' 'nonce-{SERVER_GENERATED}'; " +
  "style-src 'self' 'unsafe-inline'; " +
  "img-src 'self' data: https;; " +
  "frame-ancestors 'none'")

// 2. X-Frame-Options
// Ngăn clickjacking (iframe)
res.setHeader('X-Frame-Options', 'DENY')

// 3. X-Content-Type-Options
// Ngăn MIME type sniffing
```

Demo: JWT Inspect & Tamper

Thử thay đổi payload và xem server từ chối

```
const jwt = require('jsonwebtoken')

// Tạo token với secret
const token = jwt.sign(
  { sub: 'user_1', role: 'user' },
  'my-secret-key',
  { expiresIn: '1h' }
)
console.log('Token:', token)

// Verify: thành công
try {
  const decoded = jwt.verify(token, 'my-secret-key')
  console.log('Decoded:', decoded.role) // 'user'
} catch (err) {
  console.log('Error:', err.name)
}

// Verify với sai secret: THẤT BẠI
try {
  jwt.verify(token, 'wrong-key')
} catch (err) {
```

Tóm tắt Buổi 3

Những điểm cần nhớ

MENTAL MODEL - BUỔI 3

1. HTTP = NỀN TẢNG WEB

Methods (GET/POST/PUT/DELETE) + Status Codes + Headers

REST: Uniform Interface, Stateless, Cacheable

2. AUTHENTICATION vs AUTHORIZATION

Authn: Bạn là ai? (login)

Authz: Bạn được làm gì? (permissions)

3. SESSION vs JWT

Session: Server lưu, dễ revoke, cần store

JWT: Self-contained, stateless, khó revoke

4. CORS

Browser policy: Origin khác nhau → chặn request

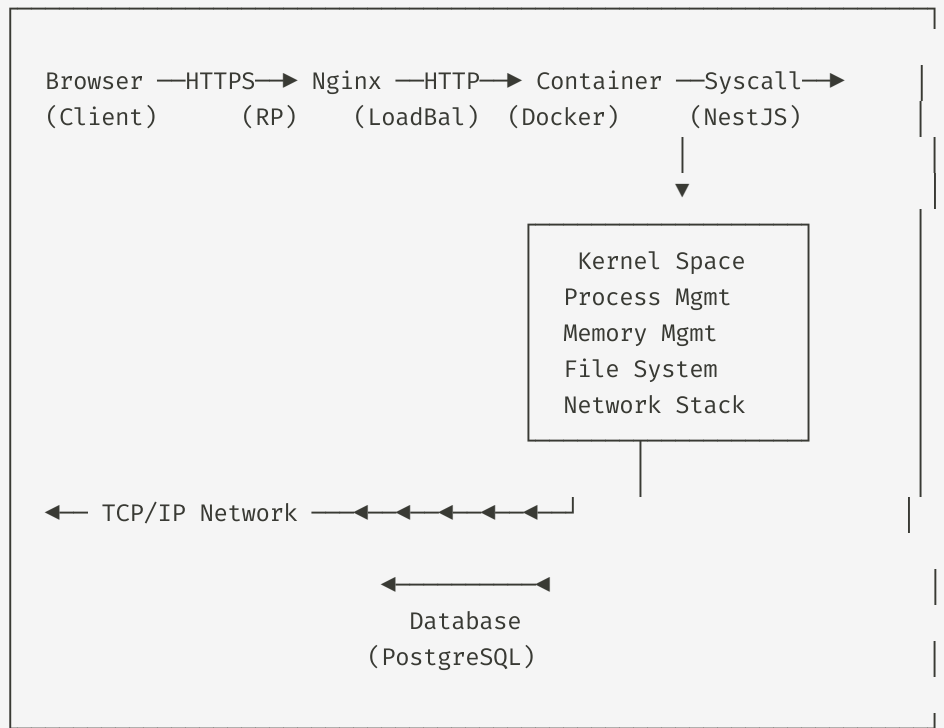
Server phải trả Access-Control-Allow-Origin

5. SQL INJECTION

Input thành SQL → DROP TABLE / bypass login

Tổng kết toàn khóa

Mental Model từ đầu đến cuối



Lộ trình tự học

Bước tiếp theo sau khóa học

Level 1 - Cần thực hành ngay:

- Viết script bash tự động hóa
- Deploy 1 app lên server (DigitalOcean, Railway)
- Viết API RESTful với Express/NestJS
- Build Dockerfile cho project hiện tại

Level 2 - Sâu hơn:

- Kubernetes để orchestrate containers
- CI/CD pipeline (GitHub Actions)
- Monitoring & Logging (Prometheus, Grafana)
- Message Queue (Redis, RabbitMQ)

Level 3 - Chuyên sâu:

Cảm ơn các bạn!

Đặt câu hỏi hoặc liên hệ

Tài liệu kèm theo:

- Cheat sheet Linux commands
- Cheat sheet Docker commands
- Cheat sheet Networking commands
- Demo code (syscall, docker, jwt)
- Mind map tổng hợp mental model

Ghi nhớ:

"Công cụ thay đổi theo năm tháng, nhưng nguyên lý tồn tại hàng thập kỷ."

Hãy hiểu **tại sao**, không chỉ **làm thế nào**.

Phụ lục: Các lệnh Linux thường dùng

Quick reference

```
# Navigation
ls -la /path          # Liệt kê file (bao gồm hidden)
cd /path              # Di chuyển
pwd                  # Thư mục hiện tại
mkdir name            # Tạo thư mục

# File operations
cp src dst            # Copy
mv src dst            # Di chuyển/đổi tên
rm -rf name           # Xóa (force, recursive)
cat file              # Xem nội dung
head -n 20 file       # 20 dòng đầu
tail -f file           # Theo dõi log realtime

# Process
ps aux                # Processes đang chạy
kill -9 PID           # Kill process
htop                  # Monitor tương tác
strace -c cmd          # Trace syscalls

# Network
curl -v url            # HTTP request chi tiết
```

Phụ lục: Các lệnh Docker thường dùng

Quick reference

```
# Image
docker build -t name:tag .      # Build image
docker images                  # Liệt kê images
docker rmi name:tag            # Xóa image
docker pull image:tag          # Pull từ registry

# Container
docker run -d -p 3000:3000 name # Chạy container
docker ps -a                  # Liệt kê containers
docker stop/rm name           # Stop/remove
docker logs -f name           # Xem logs
docker exec -it name bash      # Vào container

# Compose
docker compose up -d           # Start all
docker compose down            # Stop all
docker compose logs -f         # Logs all

# Cleanup
docker system prune -a         # Xóa tất cả unused
docker image prune              # Xóa dangling images
```